Computers & Industrial Engineering 136 (2019) 46–56

Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie

# On optimal dynamic pegging in rescheduling for new order arrival

Shokraneh K. Moghaddam[a], Kazuhiro Saitou[b,*]

[a] Department of Industrial Engineering, Sharif University of Technology, Tehran 14588-89694, Iran
[b] Department of Mechanical Engineering, University of Michigan, Ann Arbor, MI 48109-2125, USA

ARTICLE INFO

*Keywords:*
Dynamic pegging
Advanced planning and scheduling
Rescheduling
Multi-level production

ABSTRACT

A rescheduling method for new order arrival is presented based on the concept of dynamic pegging in multi-level production. Upon the arrival of new, unplanned orders, dynamic pegging enables the re-assignment of the Work-In-Progress (WIP) to the existing or newly arrived orders in such a way that the cost of rescheduling is minimized. A simple yet inspiring Mixed Integer Programming (MIP) model is proposed that combines dynamic pegging with rescheduling. The resulting schedules are compared with the ones obtained from two other basic rescheduling approaches. Analysis of the comparisons confirm the superiority of the proposed method in terms of total rescheduling costs.

## 1. Introduction

Manufacturing IT solutions such as Material Requirements Planning (MRP) and Manufacturing Resource Planning (MRPII) aim at effectively planning and managing resources of a manufacturing company. However, these solutions assume fixed lead times, infinite capacity, and backward scheduling (Billington, McClain, & Thomas, 1983). Moreover, they do not consider operation sequences. Due to ignoring capacity constraints, a feasible production schedule might not even exist for a generated production plan (Chen & Ji, 2007a, 2007b). To overcome these limitations, the concept of Advanced Planning and Scheduling (APS) that integrates production planning and scheduling, was developed (Chen & Ji, 2007a, 2007b; Steger-Jensen, Hvolby, Nielsen, & Nielsen, 2011). While planning a schedule, APS systems integrate production data and take operation sequences and capacity constraints into account. A schedule generated by APS, however, might become infeasible as new orders arrive, due dates change, or other resource/job-related disruptions happen in the manufacturing system (Ouelhadj & Petrovic, 2009). To adapt to such disruptions, dynamic scheduling techniques and re-scheduling heuristics have been proposed and were proven effective (Ouelhadj & Petrovic, 2009). While these methods can adjust the original schedules after certain disruptions occur, the post-event schedules mostly re-allocate the available resources at the time of the disrupting event, and do not take advantage of the Work-In-Progress (WIP) produced prior to that event.

This paper introduces an alternative method for schedule adjustments based on the concept of *dynamic pegging*. This concept is applicable to manufacturing systems and supply chains that operate multi-level, mixed-model production. Pegging is the information regarding which, jobs are assigned to orders. In MRP/MRPII and most APS systems, pegging is generated at the beginning of a planning cycle and remains fixed until the start of a new planning (i.e., fixed pegging). In dynamic pegging, on the other hand, the job-order assignment can change when disruptions such as arrival of new orders occur during the planning horizon. Dynamic pegging can be beneficial in multi-level production, since WIP of a job (*e.g.*, subassemblies) can be re-assigned to another order (existing or new) as long as the due dates of the unassigned orders can still be met. Dynamic pegging coupled with rescheduling can therefore be more efficient than simple rescheduling, since it takes advantage of WIP prior to the time of a specific event. While advanced commercial APS systems such as SAP's Advanced Planning and Optimization (APO) (Kallrath & Maindl, 2006) reportedly implement such capability, to the best of the authors' knowledge, no mathematical framework for optimal job-order re-assignment in dynamic pegging is found in the literature. This paper aims to fill this gap by presenting a simple yet inspiring optimization-based formulation for dynamic pegging. The formulation is intentionally made minimalistic to emphasize the characteristics of manufacturing systems where such a strategy can be most effective.

The rest of this paper is organized as follows: Section 2 presents the literature review on APS and dynamic scheduling. Section 3 introduces the optimization model and its assumptions. Section 4 presents the results of the example productions and the sensitivity analysis, in Section 5 a heuristic GA based algorithm is proposed for solving the large-scale more complex problems. Finally, Section 6 entails the conclusion and the areas for further research.

* Corresponding author.
*E-mail address:* kazu@umich.edu (K. Saitou).

https://doi.org/10.1016/j.cie.2019.07.012
Received 18 January 2019; Received in revised form 12 June 2019; Accepted 7 July 2019
Available online 09 July 2019
0360-8352/ Published by Elsevier Ltd.

## 2. Literature review

### 2.1. Integrated production planning and scheduling

Since the integrated production planning and scheduling problem, also known as the APS problem, is considered NP-hard (Sukkerd & Wuttipornpun, 2016; Wuttipornpun & Yenradee, 2014), earlier studies focused primarily on heuristic approaches (Bahl & Ritzman, 1984; Hastings, Marshall, & Willis, 1982; Taal & Wortmann, 1997), or on mathematical programming approaches with simplifying assumptions and approximations (Dillenberger, Escudero, Wollensak, & Zhang, 1994; Kolisch, 2000; Lasserre, 1992; Moon, Kim, & Gen, 2004).

Yan, Xia, Zhu, Liu, and Guo (2003) proposed three (Embedded, Alternate, and Serial) tabu-search based algorithms combined with mixed integer programming (MIP) and simulation for the production planning and scheduling problem on automobile assembly lines without splittable batches. To do so, a rough production plan is obtained and tabu-search is used iteratively afterwards for finding the near optimal production plan and schedule. Zhang and Yan (2005) formulated the integrated production planning and scheduling problem as a non-linear mixed integer program and proposed a hybrid genetic algorithm (GA) for solving it. An alternate iterative GA was also proposed by Yan, Wan, and Xiong (2015) for solving a similar non-linear MIP formulation. Yan and Zhang (2007) proposed a monolithic optimization model for production planning and scheduling in a three stage manufacturing system; a job shop, a parallel flow shop, and a single machine shop. To solve the model, a heuristic approach was implemented where a dynamic batch splitting and amalgamating procedure were introduced into GA. The simultaneous batch splitting and scheduling problem was considered by Yan, Wang, and Zhang (2013) and a GA based heuristic was used for solving both problems simultaneously. An MIP model was introduced by An and Yan (2016) for solving the integrated production planning and scheduling on synchronous automobile assembly lines. To address the difficulties in solving the model, due to the NP-hard nature of the problem, a Lagrangean relaxation method was proposed and the model was decomposed into three sub-problems namely planning, sequencing, earliness and tardiness.

While minimizing the make-span has been a common objective in these early APS studies, the wide adoption of Just-In-Time (JIT) strategy prompted the inclusion of earliness-tardiness penalties in the objective function (Cheng & Gupta, 1989; Gordon, Proth, & Chu, 2002). More recently, Chen and Ji (2007a, 2007b) proposed a MIP model for the APS problem that successfully integrates the production planning and scheduling problems. Their model takes capacity constraints, operation sequences, lead times, and due dates into account, with the objective of minimizing the cost of idle time and tardiness/earliness penalties in a multi-order environment for products with multi-level structures. Örnek, Özpeynirci, and Öztürk (2010) later extended this model by allowing an operation to be assigned to multiple eligible machines in the system. While these models are more realistic than the earlier work, they become intractable in the large scale. Many subsequent studies, therefore, presented heuristics and metaheuristics to handle the problem in real-world scenarios (Öztürk & Örnek, 2012; Sukkerd & Wuttipornpun, 2016, 2017).

### 2.2. Dynamic scheduling and rescheduling

The objectives of dynamic scheduling are maximizing productivity while minimizing deviation from the original schedule *after* a disruption, either job-related or resource-related, happens in the system. After a new event occurs, the original schedule is adjusted, through a schedule repair or rescheduling, in such a way that the make span, shop flow time, or lateness are minimized and/or system's stability is maximized (Ouelhadj & Petrovic, 2009). Depending on the way repair/rescheduling is triggered, variations such as completely reactive, robust-proactive, and predictive-reactive scheduling have been introduced (Chen & Ji, 2007a, 2007b; Hamzadayi & Yildiz, 2016; Jiang, Liu, Lin, & Zhong, 2016; Wu, Storer, & Pei-Chann, 1993). Extended high-level evaluation Petri nets (EHLEP-N) was used to make the real-time decisions about a flexible manufacturing system and to further investigate and evaluate dynamic scheduling rules (Yan, Wang, Cui, & Zhang, 1997; Yan, Wang, Zhang, & Cui, 1998). Adaptive and interoperable adaptive scheduling approaches were also proposed to address the uncertainty and dynamic nature of production systems in knowledgeable manufacturing systems (Wang & Yan, 2016; Yang & Yan, 2009).

However, in multi-level mixed-model production, taking advantage of the WIP produced prior to a disruption would be effective; like the delayed product differentiation strategy (Gupta & Benjaafar, 2004; Lee & Tang, 1997) which manages new order arrival by delaying the pegging of WIP, that are common to all models of a certain product. Instead of pegging the previously unpegged WIP at the time of a new order arrival, the dynamic pegging strategy presented in this paper, changes the original pegging at the time of a disruption.

## 3. Optimization model

This section describes an MIP model of rescheduling for new order arrival, based on the concept of dynamic pegging. This minimalistic model is designed with the intention of demonstrating the effectiveness of the dynamic pegging strategy.

### 3.1. Assumptions

Products, either in single or multiple models, are assumed to have multi-level structures represented by product structure trees. An example of a product structure tree for a single model product is shown in Fig. 1, where the end product (P1) is composed of two subassemblies (P2, P3), each of which is respectively composed of two parts (P4, P5) and (P5, P6). The edge labels (1s) indicate, for example, one P2 and one P3 are needed to produce one P1. For simplicity, we will refer to nodes in product structure trees as *items*, which represent either subassemblies (if non-terminal) or parts (if terminal).

The following assumptions are made about the production:

- In product structure trees, the processing of a parent item cannot start before the processing of all of its child items are finished. For example, the processing of P1 in Fig. 1 cannot start before the processing of both P2 and P3 are finished,
- Each item can be processed on one machine at a given time, the operations are non-preemptive, and a machine can process one item at a time,
- Machines are uninterruptedly available for eight hours per day, based on the given processing time of each item, which includes

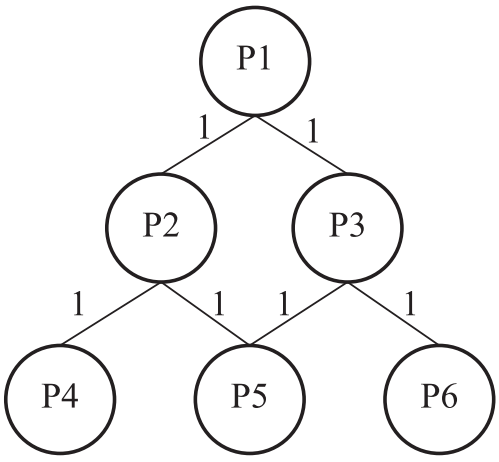



**Fig. 1.** Product structure tree of an example product.

setup and transfer time,

Also, the following assumptions are made on order arrival:

- For each order, a product model (specified by an ID in the product structure tree), quantity, due dates, and earliness/tardiness penalties are given,
- As a disruption, a "surprise" order arrives on a random day during the scheduling horizon, for which the adjustments in pegging and schedules are made in the next day,
- All original orders are placed at the beginning of the scheduling horizon.

The objective is to find an optimal schedule for all orders (original and surprise) by taking advantage of re-pegging the WIP at the time of the surprise order arrival, such that both production idle time and penalties on tardiness and earliness are minimized during the scheduling horizon.

### 3.1.1. Mathematical model

The proposed model for optimal dynamic pegging is based on Chen and Ji (2007a, 2007b), with additional parameters and decision variables to represent a surprise order and the adjustable pegging. The inputs and variables for the model are summarized below:

***Inputs:***

- $A(p)$: Set of child items of item $p$
- $A'(p)$: Set of parent items of item $p$
- $Edge_{pq}$: Number of item $p$ required for one parent item $q$
- $n$: Number of orders when the scheduling starts
- $P_i$: Item ID of the end product for order $i$
- $Q_i$: Quantity of order $i$
- $DD_i$: Due date of order $i$
- $N_{ip}$: Number of item $p$ needed to produce one product model $P_i$
- $AP_p$: Available produced quantity of item $p$ when the scheduling starts
- $TC_i$: Tardiness cost per day for order $i$
- $EC_i$: Earliness cost per day for order $i$
- $m$: Number of machines
- $t_{ipk}$: Processing time required by item $p$ ($p = 1, \ldots, P_i$) of order $i$ on machine $k$ ($k = 1, \ldots, m$)
- $I$: Cost of idle time per hour
- $\alpha$: A large positive number

***Dependent variable:***

- $NP_{ip}$: Quantity of item $p$ that is not required to be produced for order $i$ due to the assignment of item $p$'s parent to order $i$.

***Independent variables:***

- $C_{max}$: Production make span
- $C_i$: Production completion time of order $i$
- $L_i$: Number of tardy days (real number) for order $i$
- $E_i$: Number of Early days (real number) for order $i$
- $L_i^I$: Number of tardy days (integer) for order $i$
- $E_i^I$: Number of Early days (integer) for order $i$
- $S_{ipk}$: Production start time of item $p$ of order $i$ on machine $k$
- $x_{ipjqk}$: 1, if item $p$ of order $i$ proceeds item $q$ of order $j$ on machine $k$; 0, otherwise
- $y_{ip}$: Available produced quantity of item $p$ assigned to order $i$ when the scheduling starts

The model is given as a mixed-integer programming model shown in (1). In the optimization model in Eq. (1), the objective is to minimize Eq. (1.1), which is the sum of the idle time cost of the machines as well as the tardiness/earliness penalties of the orders. Constraint (1.2) indicates that the production completion time of each order must not be greater than the production make span. Constraint (1.3) ensures that the processing of an item pegged to an order on a machine can only start after processing the required quantity of its child item, pegged to the same order on any machine. Constraint (1.4) indicates that the completion time of an order is equal to the start time of the end-product plus the processing time of the required quantity of that end-product. In constraints (1.3), (1.4), the required quantity of a specific order (after the arrival of surprise order) might be fewer than the original order amount, as some common items which have already been produced before the disruption, might later be (dynamically) pegged to that order. Constraints (1.5) and (1.6) are common constraints in scheduling problems ensuring that no two operations are processed on a machine simultaneously. The above remarks on the required quantity of an order apply to constraint (1.5) as well.

Constraints (1.7)–(1.9) are the ones concerned with dynamic pegging, i.e., the assignment of previously pegged and produced items to different orders. Constraint (1.7) indicates that the total quantity of an item, pegged to different orders, cannot exceed the available produced quantity of the item. Constraint (1.8) indicates that the total quantity of an item pegged to an order should not exceed the required quantity of the item for the order. Constraint (1.9) indicates, if an available produced item is pegged to an order, all of its child items are also pegged to the order. In other words, if a parent item is pegged to an order, its child items are no longer needed to be produced. Constraints (1.10) and (1.11) measure the tardiness/earliness respectively. Since each day is assumed to be eight (8) hours, the completion time of each operation is divided by eight. Also, all the penalty costs are in the unit of days, hence constraints (1.12) and (1.13) convert tardiness and earliness values to integers. Finally, constraints (1.14)–(1.18) define the domains of the independent variables.

$$min \left\{ I \times \left( m \times C_{max} - \sum_{k=1}^{m} \sum_{i=1}^{n} \sum_{p=1}^{P_i} (t_{ipk} \times N_{ip} \times Q_i) \right) + \sum_{i=1}^{n} [TC_i \times L_i^I + EC_i \times E_i^I] \right\} \quad (1.1)$$

subject to:

$$C_i \leqslant C_{max} \quad \forall\, i \quad (1.2)$$

$$S_{ipl} \geqslant S_{iqk} + t_{iqk} \times (N_{iq} \times Q_i - y_{iq} - NP_{iq}) \quad \forall\, q \in A(p), \forall\, i, p, k, l, \quad (1.3)$$

$$S_{iP_ik} + t_{iP_ik} \times (Q_i - y_{iP_i}) = C_i \quad \forall\, i, k \quad (1.4)$$

$$S_{ipk} \geqslant S_{jqk} + t_{jqk} \times (N_{jq} \times Q_j - y_{jq} - NP_{jq}) - \alpha(x_{ipjqk}) \quad \forall\, i, j, p, q, k \quad (1.5)$$

$$x_{ipjqk} + x_{jqipk} = 1 \quad \forall\, i, j, p, q, k \quad (1.6)$$

$$\sum_{i=1}^{n} (y_{ip} + NP_{ip}) = AP_p \quad \forall\, p \quad (1.7)$$

$$y_{ip} + NP_{ip} \leqslant N_{ip} \times Q_i \quad \forall\, i, p \quad (1.8)$$

$$NP_{ip} = \sum_{q \in A'(p)} y_{iq} \times Edge_{pq} \quad \forall\, i, p \quad (1.9)$$

$$\frac{C_i}{8} - DD_i \leqslant L_i \quad \forall\, i \quad (1.10)$$

$$DD_i - \frac{C_i}{8} \leqslant E_i \quad \forall\, i \quad (1.11)$$

$$L_i^I \geqslant L_i \quad \forall\, i \quad (1.12)$$

$$E_i^I \geqslant E_i - 0.99 \quad \forall\, i \quad (1.13)$$

$$C_{max} \geqslant 0 \quad (1.14)$$

$$S_{ipk} \geqslant 0 \quad \forall\, i, p, k \quad (1.15)$$

$$C_i, L_i, E_i \geqslant 0 \quad \forall\, i \quad (1.16)$$

$$L_i^I, E_i^I, y_{ip} \in \mathbb{Z}_+^n \quad \forall\, i, p \quad (1.17)$$

$$x_{ipjqk} \in \{0, 1\} \quad \forall\, i, j, p, q, k \quad (1.18)$$

Using the model in Eq. (1), the optimal dynamic pegging problem can be solved with the following three steps, which will be referred to as the repegging-rescheduling (RPRS) approach through rest of the paper:

- **Step 1:** solve model (1) with no available produced items (i.e., $AP_p = 0$, $\forall p$) for the entire scheduling period, to obtain the *original schedule*. It is equivalent to solving the model presented by Chen and Ji (2007a, 2007b).
- **Step 2:** calculate available produced items $AP_p$ at the time of the surprise order arrival, according to the original schedule obtained in Step 1. If the surprise order arrives during the production of an item, the produced quantity of the item would be rounded to the nearest smaller integer.
- **Step 3:** solve model (1) with $AP_p$ obtained in Step 2 for the period starting at the begging of the next day of the surprise order arrival, to obtain the *dynamically-pegged, adjusted schedule*. Note that the due dates for the original orders should be updated to reflect the new scheduling period.

## 4. Results

### 4.1. Simple example

This section presents a simple example based on the product presented in Fig. 1, to illustrate the application of the proposed RPRS approach. Table 1 shows the processing time of each item on two different machines. Cost of idle time is \$6/hour. There is only one original order; 5 P1's due in 3 days with tardiness cost of \$8/day and earliness cost of \$4/day. During the first day of production (during the first 8 h), a surprise order arrives; 3 P1's due in 1 day with the same earliness/tardiness costs as the original order. The solutions for Steps 1 and 3 were obtained in less than a second with GAMS v.24.1.3 software, on a 2.6 GHz Intel® Core(TM) i7-6500U CPU system.

**Step 1:** Fig. 2 shows the obtained original schedule as a Gantt chart on each machine. The numbers in each slot represent the order number followed by the item number (*i.e.*, pegging). The total cost and the total make span are \$60 and 17.5 h, respectively, and the order has met its due date.

**Step 2:** At the end of the first day, 3.5 P2's (which is rounded to 3), 5 P4's, 10 P5's, and 5 P6's are produced. Since items P4 and P5 are child items of P2, 3 P4's and 3 P5's are used for producing 3 P2's. Therefore, $AP$ is calculated as: $AP_2 = 3$, $AP_4 = 5–3 = 2$, $AP_5 = 10–3 = 7$, and $AP_6 = 5$.

**Step 3:** Since the rescheduling occurs at the beginning of the second day, the due date of the original order is updated to 3–1 = 2 days. Fig. 3 shows the dynamically-pegged, adjusted schedule. The adjusted pegging for each item produced before the second day is shown in the slots on the Gantt chart before the zero hour (in the grayed region). Except for item P6, portions of other produced items pegged to the original order in the original schedule are re-pegged to the surprise order. The total cost and the total make span are \$32 and 13 h, respectively, for the reschedule period. The original order is met its due date while the surprise order is met 1 day after its due date.

**Table 1**
Machine processing times for the items.

| Items | Machine number | Processing time (hours) |
|---|---|---|
| P1 | M1 | 1 |
| P2 | M1 | 1 |
| P3 | M2 | 1 |
| P4 | M1 | 0.5 |
| P5 | M2 | 0.5 |
| P6 | M2 | 0.5 |

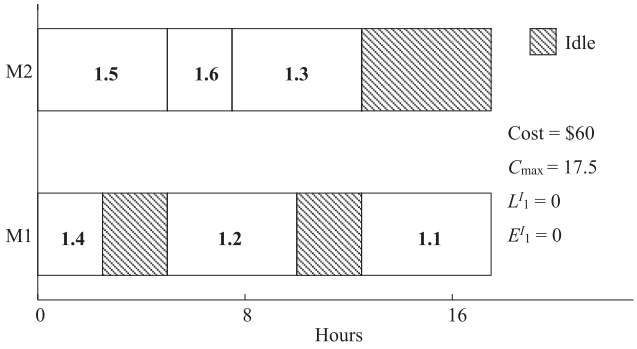



**Fig. 2.** Original schedule of the simple example.

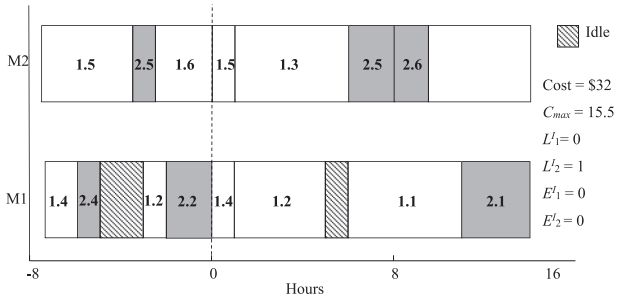



**Fig. 3.** Dynamically-pegged, adjusted schedule of the simple example after the surprise order arrival.

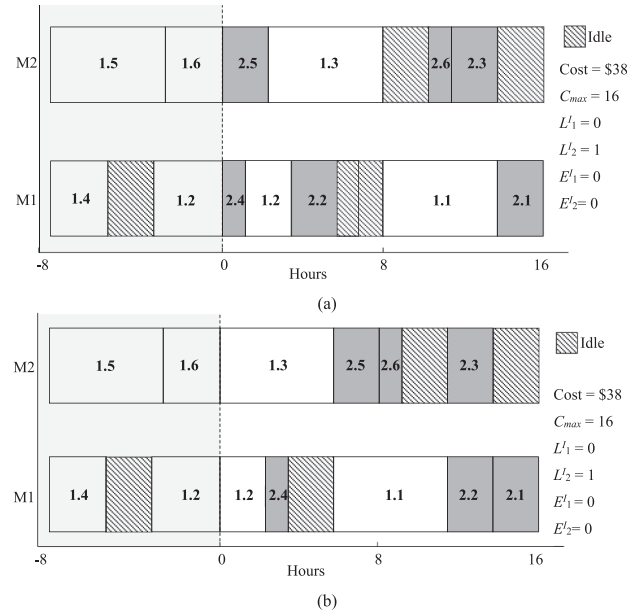



**Fig. 4.** Schedule of the simple example after the surprise order arrival: (a) FPRS and (b) FPFS.

**Table 2**
Parameter values for the sensitivity analysis of the simple example.

| Parameters | Values |
|---|---|
| Q (quantity) | 1, 2, 3 |
| DD (due date) | 1, 2, 3 |
| EC (earliness cost) | 1, 4, 8, 20, 50 |
| TC (tardiness cost) | 8, 10, 20, 50, 100 |

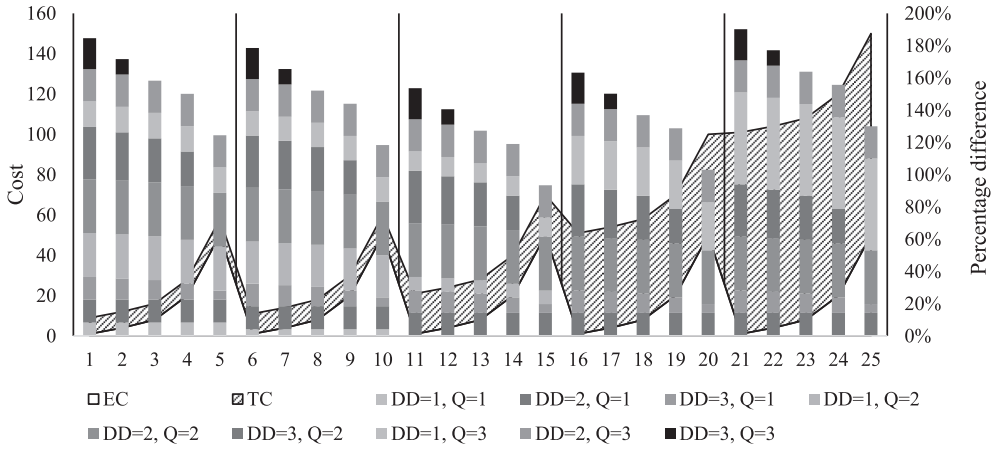



**Fig. 5.** Percentage difference between the costs of FPRS and RPRS results.

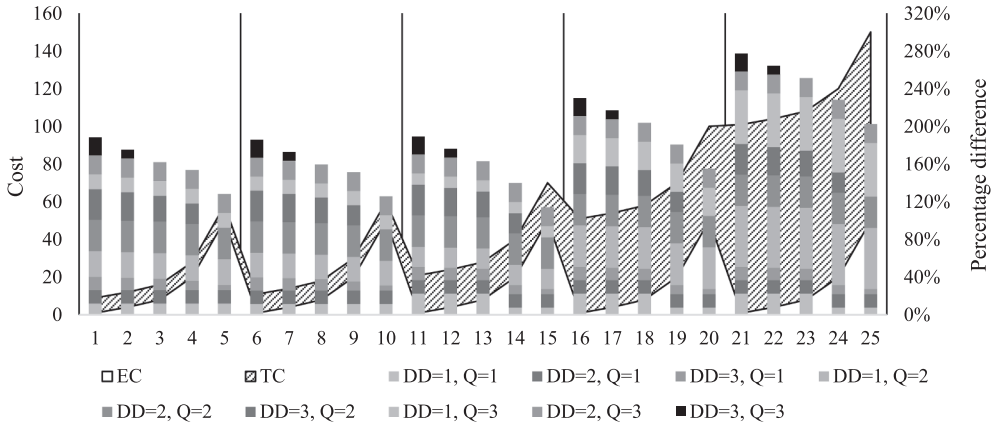



**Fig. 6.** Percentage difference between the costs of FPFS and RPRS results.

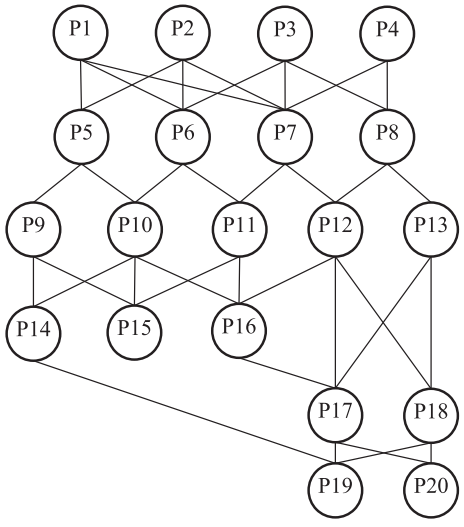



**Fig. 7.** Product structure tree of the second example.

### 4.2. Simple example: comparison cases

To demonstrate the benefits of the proposed RPRS approach, the simple example in the previous section is solved using two different approaches for comparison: fixed pegging-rescheduling (FPRS) and fixed pegging-fixed schedule (FPFS). In FPRS, upon the arrival of a surprise order, a new schedule for all orders (original and surprise) is created based on the pegging of the original schedule. In FPFS, upon the arrival of a surprise order, a new schedule is created for the surprise order only, based on the idle time of machines in the original schedule.

The results are shown in Fig. 4. In both FPRS and FPFS approaches the total cost and the total make span during the reschedule period are $38 (19% higher than RPRS) and 16 h (half an hour longer than RPRS), respectively. The original order has met its due date while the surprise order is met 1 day after its due date (same as RPRS). In fact, it is expected that RPRS produces no worse solutions than FPRS and FPFS; mathematically, the proposed RPRS is a relaxation of FPRS, and FPRS itself is a relaxation of FPFS as well. According to the above statement, any rescheduling method in the literature which is based on the fixed pegging approach can benefit from incorporating the concept of dynamic pegging in RPRS as well.

**Table 3**
Machine processing times for the items in the second illustrated example.

| Item | P1 | P2 | P3 | P4 | P5 | P6 | P7 | P8 | P9 | P10 | P11 | P12 | P13 | P14 | P15 | P16 | P17 | P18 | P19 | P20 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Machine# | M1 | M2 | M3 | M4 | M1 | M2 | M3 | M4 | M1 | M2 | M3 | M4 | M4 | M1 | M2 | M3 | M3 | M4 | M1 | M2 |
| Processing time (hours) | 0.5 | 0.5 | 0.5 | 0.5 | 0.4 | 0.4 | 0.4 | 0.4 | 0.3 | 0.3 | 0.3 | 0.3 | 0.3 | 0.4 | 0.4 | 0.4 | 0.3 | 0.3 | 0.2 | 0.2 |

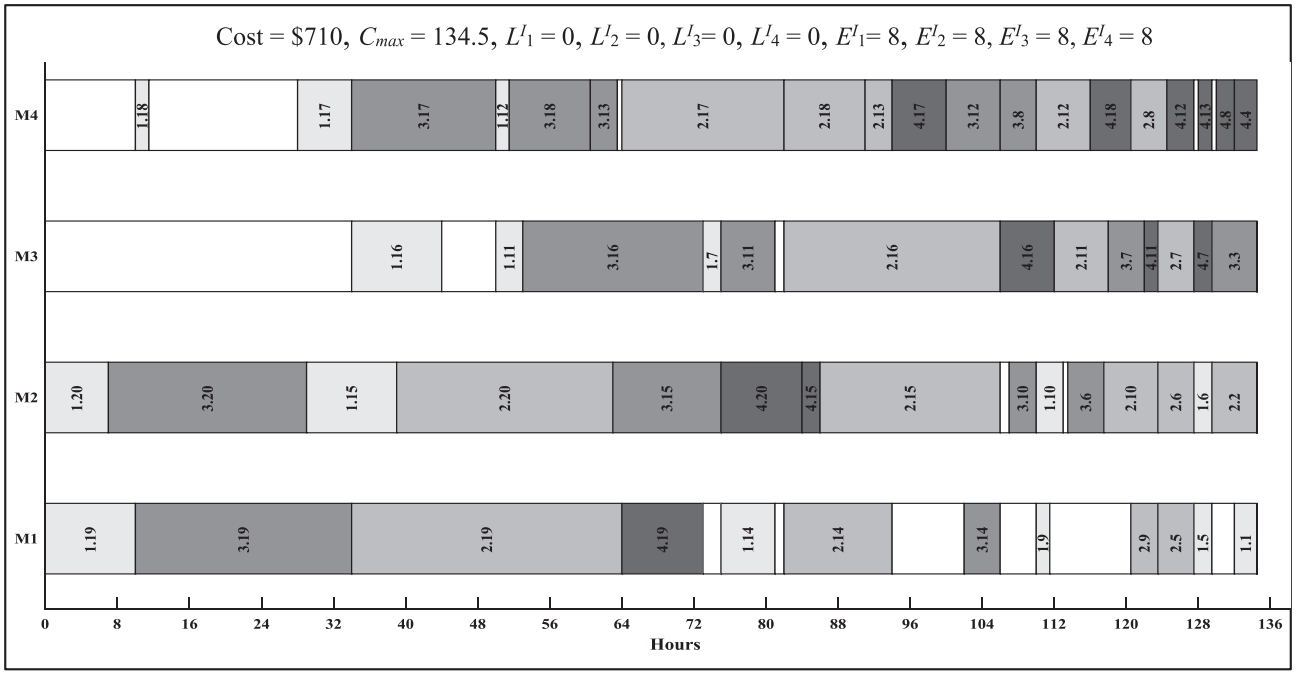



**Fig. 8.** Original schedule of the second example.

### 4.3. Simple example: sensitivity analysis

To gain insights on characteristics of the problems where RPRS is most effective, a sensitivity analysis is performed. The simple example is solved for all combinations of parameter values associated with the surprise order shown in Table 2.

Fig. 5 shows the percentage difference between the costs of FPRS and RPRS results, plotted for different combinations of the values of EC and TC in Table 2. The stacked area behind the bars represents the total of 5 X 5 = 25 combinations of EC and TC, labeled as 1–25 on the horizontal axis. For each combination, the corresponding stacked bar represents the percentage difference between FPRS and RPRS costs for different values of DD and Q. For example, the solid black portion of the stacked bars represent cases where the values of DD and Q are both 3.

It is observed that the difference between the costs is most significant (as large as 57%) when the surprise order has relatively large quantity, tight due date, and high tardiness cost. Also, the cost difference tends to be smaller (not always) when the difference between earliness and tardiness costs are smaller. Fig. 6 illustrates the percentage difference between the costs of FPFS and RPRS results, where overall trends similar to Fig. 5 are observed with larger differences (as large as 65%).

### 4.4. Mixed model production example

This section presents the second, more complex example, where the mixed model production of four different end products is scheduled with the proposed RPRS approach. Fig. 7 shows the product structure tree consisting of 20 items including 4 end products (P1-P4). While not shown, it is assumed each parent item requires only one of every child item to be produced. There are 4 machines available in the production system and the processing time of each item is given in Table 3.

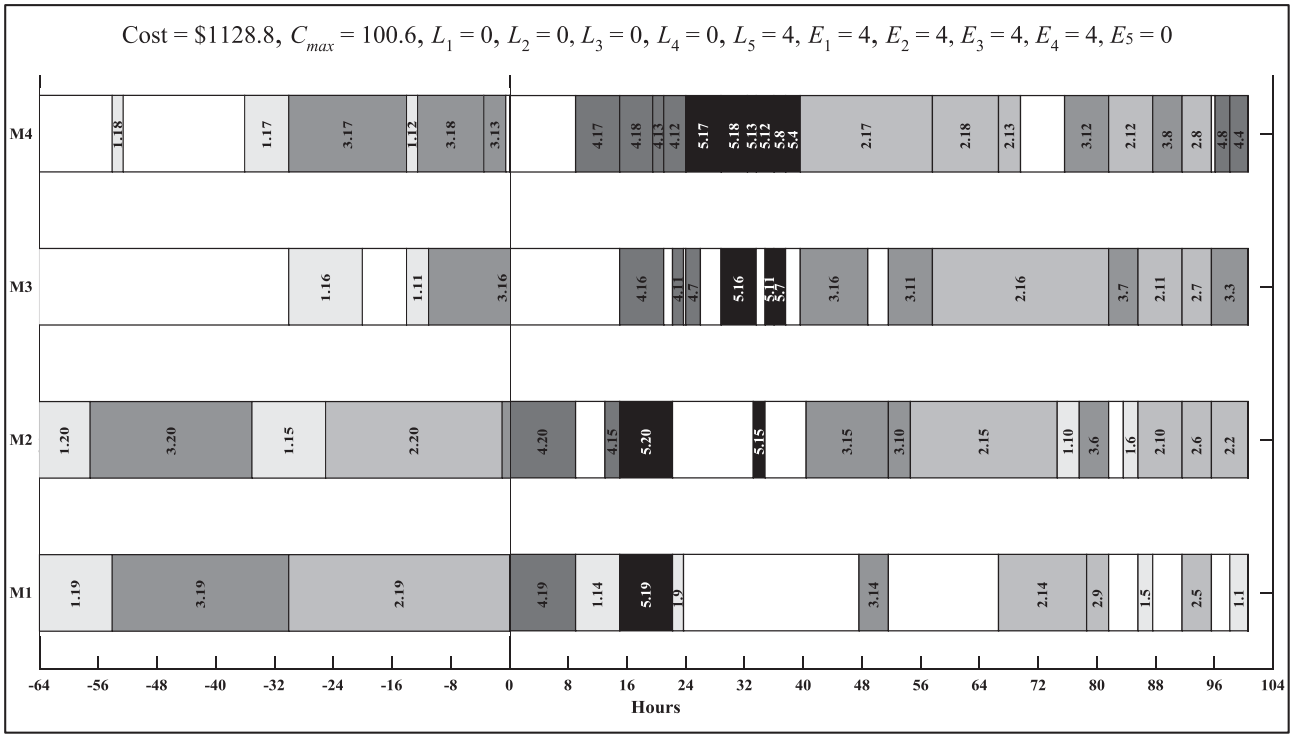
There are four (4) original orders; quantities for P1, P2, P3, and P4 are 5, 10, 10, and 5 respectively. All orders are due in 25 days with tardiness cost of $8/day and earliness cost of $4/day. During the 8th day of production, a surprise order arrives; 4 P4's due in 1 day with tardiness cost of $100/day and earliness cost of $4/day. Based on the findings in Section 4.3, the tight due date and high tardiness cost are chosen to demonstrate the effectiveness of the proposed RPRS approach. Also, in order to increase the opportunity for re-pegging, the arrival of the surprise order, in this example, occurs late enough (i.e., 8 days after the start of the original production schedule). The solution for Step 1 was obtained in 6413.406 s in the same computing environment as the simple example. However, due to complexity of the mixed model production, the optimal solution for Step 3 could not be found after 9000 s and the best obtained solution up to that moment is reported instead.

Fig. 8 shows the obtained original schedule as a Gantt chart on each machine. The idle time slots are shown in white. The total cost and the total make span are $710 and 134.5 h (roughly 17 days), respectively. All orders are completed 8 days before their due dates.

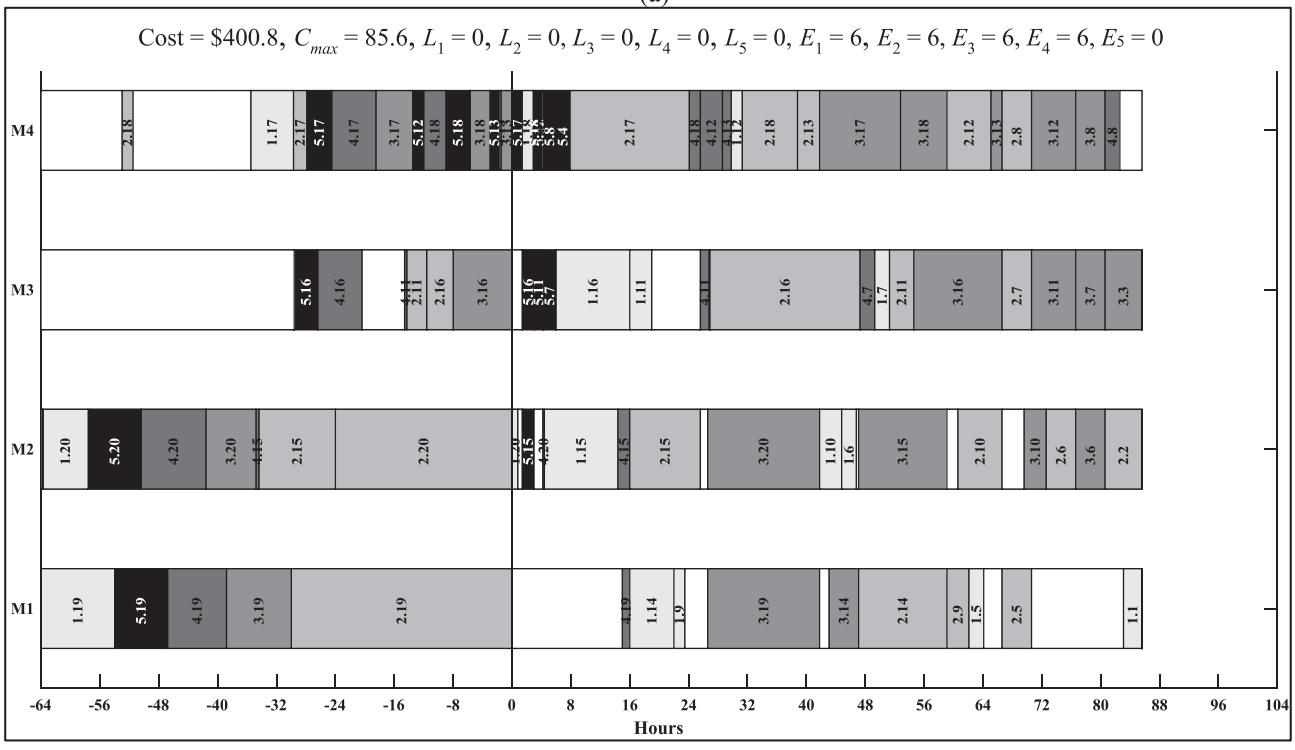
Fig. 9(a)-(b) shows the adjusted schedules after the surprise order arrival, using the FPRS and RPRS approaches, respectively. The FPRS schedule was obtained after 565.2 s, has a duration of 100.6 h and is 181.6% more expensive than the RPRS schedule. In the RPRS schedule, the completion time is 85.6 h (15 h less than FPRS) and all orders, original and surprise, have met their due dates, whereas the FPRS schedule results in a 4-day delay for the surprise order. As seen in Fig. 9(b), most of the produced items before the surprise order arrival ended up reassigned to the surprise order, as it has a tight due date of 1 day and a high tardiness cost of $100. The FPFS approach was not attempted in this example since FPFS is expected to produce no better result than FPRS.

## 5. The heuristic algorithm based on GA

Due to complexity of the proposed model and the exponential running time of available algorithms (i.e., branch and bound) for large-scale advanced planning and scheduling problems (such as the one presented in Section 4.4), heuristic algorithms are often designed and




(a)




(b)

**Fig. 9.** Schedule of the second example after the surprise order arrival: (a) FPRS and (b) RPRS.

| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| *Part number* | 6 | 5 | 4 | 6 | 4 | 5 | 3 | 3 | 2 | 1 | 2 | 1 |
| *Order number* | 1 | 1 | 1 | 2 | 2 | 2 | 1 | 2 | 1 | 1 | 2 | 2 |

**Fig. 10.** Chromosome representation of the simple example.

implemented. Many of the previous studies have successfully used GA to solve complicated problems of this sort, since this algorithm is proved to converge to the global optimum point (Chen & Ji, 2007a, 2007b; Sukkerd & Wuttipornpun, 2016; Yan et al., 2015; Yan et al., 2013; Zhang & Yan, 2005).

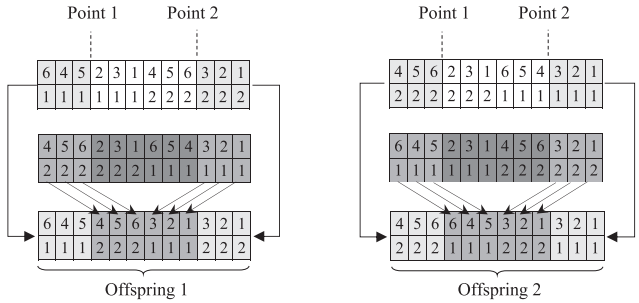



**Fig. 11.** Two-point crossover operator.

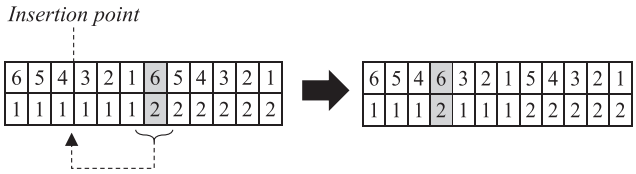



**Fig. 12.** The single point mutation operator.

In our proposed model, after a certain disruption occurs in form of a new order arrival, production planning, scheduling, and assignment of the previously produced items to all orders (new and old) must be considered simultaneously. Therefore, GA is most useful during the third step of our proposed procedure in Section 3.1.1. This step is considerably time consuming as for each specific assignment of available produced items to orders, a new production plan and schedule is required. Since the model presented in (1) is a novel approach towards solving APS problems, specific general coding methods of GA (i.e., chromosome representation and genetic operators) may no longer be applicable and making certain changes is inevitable.

### 5.1. Representation

In our proposed GA, as shown in Fig. 10, a feasible production plan is represented by an array that consists of two rows. The first row indicates the part numbers and the second row represents the corresponding order number of each part. Since the considered products in this study have multi-level structures, part numbers can't be randomly chosen for each gene of the chromosome and the precedence constrains on sub-assemblies and final products must be considered as well. Therefore, for each order, those subassemblies with no child items or those whose child items have previously been selected, fill the part number row of the array respectively.

### 5.2. Evaluation function

Each chromosome represents a valid production plan consisting of part numbers and order numbers that must be processed consecutively. Having the production plan, (1.1) can be used to evaluate the fitness of each chromosome by calculating the production start time of each part number and its corresponding production duration. Since the production plan is determined in advance, Algorithm 1 is implemented for evaluating the fitness of each production plan by calculating the process finish time of each order, the total idle times on the machines, and the tardiness/earliness penalties associated with each order.

**Algorithm 1.**

**Input:** $A(p)$, $n$, $Q_i$, $DD_i$, $N_{ip}$, $AP_p$, $TCi$, $ECi$, $m$, $t_{ipk}$, $I$, $y_{ip}$, $NP_{ip}$ (as defined in the model inputs section); One Chromosome;
**Output:** Solution (Eq. (1.1));

**Begin**
  $m(i) \leftarrow \emptyset$; // $m(i)$ is the machine associated with part $i$
  $S(i, j) \leftarrow \emptyset$; // $S(i, j)$ is the process start time of part $i$ of order $j$
  $F(i, j) \leftarrow \emptyset$; // $F(i, j)$ is the process finish time of part $i$ of order $j$
  **For** $i = 1$ **to** $LC$ **do** // LC is the length of the chromosome
    **If** $A(p_i) = \emptyset$ // $p_i$ is the part associated with gene $i$
      $S(p_i, n_i) \leftarrow$ last process finish time on $m(p_i)$; // $n_i$ is the order associated with gene $i$
    **Else**
      $S(p_i, n_i) \leftarrow \max\{\text{last process finish time on } m(p_i), \max\{F(A(p_i), n_i)\}\}$;
    **End**
    $F(p_i, n_i) \leftarrow S(p_i, n_i) + t(n_i, p_i, m(p_i)) \times [Q(n_i) \times N(n_i, p_i) - y(n_i, p_i) - NP(n_i, p_i)]$;
    Update the last process finish time on $m(p_i)$;
  **End**
  Calculate $TCi$ and $ECi$ for all orders based on the $F(i, j)$ of the final product in each order and $DD_i$;
  Calculate $I$;
  Calculate $C_{max}$ based on the finish time of all orders;
  Calculate Eq. (1.1) based on the available information;
**End**

### 5.3. Selection

To select chromosomes for reproduction, the roulette wheel strategy is used where the chromosome's selection probability $p(i)$ is directly proportional to its fitness value.

$$p(i) = \frac{f(i)}{\sum_{i=1}^{n} f(i)} \quad i = 1, \ldots, n = \textit{Population size}$$

The random number $r$ is sampled from a uniform distribution and then compared with the cumulative probability $q(k) = \sum_{i=1}^{k} p(i)$. If $r$ is not more than $q(i)$, the first chromosome is selected. Otherwise, chromosome $k$ is selected ($2 \leqslant k \leqslant n$) when $q(k-1) < r \leqslant q(k)$.

### 5.4. Genetic operators

Since correcting an infeasible solution requires time and effort, the genetic operators must perform in such a way that a certain feasible solution remains feasible throughout the GA process. This means that crossover and mutation operators must either guarantee that precedent constraints are not removed while changing the structure of chromosomes or minimize the required correction operations otherwise.

#### 5.4.1. Crossover

A two-point crossover is used as shown in Fig. 11. In this operator, two points are randomly selected. The elements outside the selected two points are inherited from one parent of the child, and the other elements are placed in order of their appearance in the other parent. This type of crossover helps maintaining the feasibility of the solutions and therefore correction operations will no longer be required.

#### 5.4.2. Mutation

Single point mutation is used in this study as shown in Fig. 12. A gene is randomly selected in a chromosome and later inserted in a random point which is chosen from a list of possible positions. A possible insertion point for a gene is a position where no precedent constraint is violated. For example, in Fig. 12, the selected gene can be




**Fig. 13.** Comparison of the GA heuristic results by changing different parameters.

inserted between any of its previous genes without violating any constraints.

### 5.5. Algorithm

Based on the above-mentioned details about the proposed GA, Algorithm 2 is used to solve the model presented in (1) subject to constraints (1.1) − (1.18), after a new order arrival. To generate the initial population, for each gene in a chromosome, a random order is selected and then a random part of the order is chosen in such a way that no precedence constraints are violated. The genes are created from left to right until all parts in all orders are assigned to specific genes.

As discussed earlier, what makes the production planning and scheduling formulation in our study even more challenging is considering the available WIP and optimally assigning it to the orders at hand. In the proposed heuristic approach, the available produced items are assigned to orders based on Algorithm 3. For each part type, a random permutation of orders is created, and produced parts of that specific type (if available) are assigned to each order based on the created permutation and the order quantity.

**Algorithm 2.**

**Input:** Model data (all inputs of the mathematical model (1))
GA parameters: $N_{pop}$ (Population size), $P_C$ (Crossover probability), $P_M$ (Permutation probability)
**Output:** Solution (a feasible production plan and schedule)

```
Begin
  P ← Generate an initial population with N_pop solutions;
  Evaluate each solution in P using Algorithm 1;
  P_best ← Best solution in P
  While (stopping criterion is not met) do
    Assign the available produced items to orders based on Algorithm 3;
    While (stopping criterion is not met) do
      P′←perform crossover operations on selected chromosomes and update the population;
      P′←perform mutation operations on selected chromosomes and update the population;
      Evaluate each solution in P′ using Algorithm 1;
      Find the best solution in P′ and update P_best;
    End
  End
End
```

**Algorithm 3.**

**Input:** Available produced items
**Output:** Solution (an assignment of produced items to the orders)

```
Begin
Assign (i, j) ← ∅;
For i = 1 to p do      // p is the total number of part types
  per ← a random permutation of orders;
  For j = 1 to n do      // n is the total number of orders
    If AP (i) > 0      // AP (i) is the available produced amount of item i
      If Q (per (j)) × N (per (j), i) ≥ Ap (i)      // Q and N are defined in the inputs of the model
        Assign (i, per (j)) ← Ap (i);
        AP (i) ← 0;
      Else
        Assign (i, per (j)) ← Q (per (j)) × N (per (j), i);
        AP (i) ← AP (i) − Q (per (j)) × N (per (j), i);
      End
    End
  End
End
End
```

### 5.6. Analysis of the results

The proposed GA-based heuristic approach is implemented to solve the mixed model production example in Section 4.4. Due to the nature of the GA, the obtained solutions are sensitive to many parameters. Here, the results are evaluated by changing the crossover and mutation probabilities, population size, the evolution generation number (*GALoop*), and the total number of different assignments that can be considered for allocating WIP to different orders (*AssignLoop*).

Fig. 13-(a, c, e, g, i) demonstrate how changing each of the mentioned parameters affect the quality of the final solutions. While increasing certain parameters such as *GALoop* and *AssignLoop* improve the solutions' quality (on average), the gap between the best solution found with the exact algorithm (400.2) and the best solution obtained from GA (542.4) is still quite significant.

In the mixed model production example of Section 4.4, the difference between the solutions obtained from FPRS and RPRS approaches lies in optimally re-assigning the available WIP and rescheduling the plans afterwards. Therefore, by changing Algorithm 3 so that parts are not randomly assigned to orders, the heuristic can be improved. To test this hypothesis, Algorithm 3 was used in the GA with a minor change. The orders were prioritized based on their tardiness penalties and due dates. The available produced items were first assigned to orders with the highest tardiness penalty and the shortest due date (in the case of the presented example, order number 5). The GA was tested under new circumstances with the same parameter settings. The best solution found through this approach equals 484.4 and the results, as shown in Fig. 13-(b, d, f, h, j), indicate an average improved quality.

## 6. Conclusion and future work

This paper proposed a new method of rescheduling for new order arrival, based on the concept of *dynamic pegging* in multi-level production. Upon the arrival of new, unplanned orders, dynamic pegging enables the re-assignment of the WIP to the existing or newly arrived orders in such a way that the cost of rescheduling is minimized. A simple MIP model is proposed that combines dynamic pegging with rescheduling. The resulting schedules are compared with the ones obtained from two other basic rescheduling approaches. Analysis of the comparisons confirm the superiority of the proposed method in terms of total rescheduling costs.

As the first reporting of this sort, it is the authors' intention to present a simple mathematical model that would effectively capture the dynamic pegging concept, with the hope that it would inspire further research that fully explore the richness of the associated problems. Such future work would include the studies on the influence of randomness in order arrival and proactive inventory planning, the integration of dynamic pegging with more advanced rescheduling techniques in the literature, the effect of other types of disruptions in the system as a trigger for rescheduling, the formulation of the yes/no decision problem of new order acceptance, and relatedly, the reduction in computational time as well as improvement in quality of the obtained solutions by better heuristics, metaheuristics, or artificial intelligence approaches.

## Acknowledgements

The work described in this paper was conducted during the first author's visit to the University of Michigan. The funding for the visit was provided by Sharif University of Technology and Iran's Ministry of Science. These sources of support are gratefully acknowledged.

## References

An, Y.-W., & Yan, H.-S. (2016). Lagrangean relaxation approach to joint optimization for production planning and scheduling of synchronous assembly lines. *International Journal of Production Research, 54*(22), 6718–6735.
Bahl, H. C., & Ritzman, L. P. (1984). An integrated model for master scheduling, lot sizing and capacity requirements planning. *Journal of the Operational Research Society,* 389–399.
Billington, P. J., McClain, J. O., & Thomas, L. J. (1983). Mathematical programming approaches to capacity-constrained MRP systems: Review, formulation and problem reduction. *Management Science, 29*(10), 1126–1141.
Chen, K., & Ji, P. (2007a). A genetic algorithm for dynamic advanced planning and scheduling (DAPS) with a frozen interval. *Expert Systems with Applications, 33*(4), 1004–1010.
Chen, K., & Ji, P. (2007b). A mixed integer programming model for advanced planning and scheduling (APS). *European Journal of Operational Research, 181*(1), 515–522.
Cheng, T., & Gupta, M. (1989). Survey of scheduling research involving due date determination decisions. *European Journal of Operational Research, 38*(2), 156–166.
Dillenberger, C., Escudero, L. F., Wollensak, A., & Zhang, W. (1994). On practical resource allocation for production planning and scheduling with period overlapping setups. *European Journal of Operational Research, 75*(2), 275–286.
Gordon, V., Proth, J.-M., & Chu, C. (2002). A survey of the state-of-the-art of common due date assignment and scheduling research. *European Journal of Operational Research, 139*(1), 1–25.
Gupta, D., & Benjaafar, S. (2004). Make-to-order, make-to-stock, or delay product differentiation? A common framework for modeling and analysis. *IIE Transactions, 36*(6), 529–546.
Hamzadayi, A., & Yildiz, G. (2016). Event driven strategy based complete rescheduling approaches for dynamic m identical parallel machines scheduling problem with a common server. *Computers & Industrial Engineering, 91*, 66–84.
Hastings, N. A., Marshall, P., & Willis, R. J. (1982). Schedule based MRP: An integrated approach to production scheduling and material requirements planning. *Journal of the Operational Research Society,* 1021–1029.
Jiang, S.-L., Liu, M., Lin, J.-H., & Zhong, H.-X. (2016). A prediction-based online soft

scheduling algorithm for the real-world steelmaking-continuous casting production. *Knowledge-Based Systems, 111*, 159–172.
Kallrath, J., & Maindl, T. I. (2006). *Real optimization with SAP® APO.* Springer Science & Business Media.
Kolisch, R. (2000). Integration of assembly and fabrication for make-to-order production. *International Journal of Production Economics, 68*(3), 287–306.
Lasserre, J.-B. (1992). An integrated model for job-shop planning and scheduling. *Management Science, 38*(8), 1201–1211.
Lee, H. L., & Tang, C. S. (1997). Modelling the costs and benefits of delayed product differentiation. *Management Science, 43*(1), 40–53.
Moon, C., Kim, J., & Gen, M. (2004). Advanced planning and scheduling based on precedence and resource constraints for e-plant chains. *International Journal of Production Research, 42*(15), 2941–2955.
Örnek, A., Özpeynirci, S., & Öztürk, C. (2010). A note on "A mixed integer programming model for advanced planning and scheduling (APS)". *European Journal of Operational Research, 203*(3), 784–785.
Ouelhadj, D., & Petrovic, S. (2009). A survey of dynamic scheduling in manufacturing systems. *Journal of Scheduling, 12*(4), 417–431.
Öztürk, C., & Örnek, A. M. (2012). A MIP based heuristic for capacitated MRP systems. *Computers & Industrial Engineering, 63*(4), 926–942.
Steger-Jensen, K., Hvolby, H.-H., Nielsen, P., & Nielsen, I. (2011). Advanced planning and scheduling technology. *Production Planning & Control, 22*(8), 800–808.
Sukkerd, W., & Wuttipornpun, T. (2016). Hybrid genetic algorithm and tabu search for finite capacity material requirement planning system in flexible flow shop with assembly operations. *Computers & Industrial Engineering, 97*, 157–169.
Sukkerd, W., & Wuttipornpun, T. (2017). Non-population search algorithms for capacitated material requirement planning in multi-stage assembly flow shop with alternative machines. *World Academy of Science, Engineering and Technology, International Journal of Mechanical, Aerospace, Industrial, Mechatronic and Manufacturing Engineering, 11*(3), 636–643.
Taal, M., & Wortmann, J. C. (1997). Integrating MRP and finite capacity planning. *Production Planning & Control, 8*(3), 245–254.
Wang, H.-X., & Yan, H.-S. (2016). An interoperable adaptive scheduling strategy for knowledgeable manufacturing based on SMGWQ-learning. *Journal of Intelligent Manufacturing, 27*(5), 1085–1095.
Wu, S. D., Storer, R. H., & Pei-Chann, C. (1993). One-machine rescheduling heuristics with efficiency and stability as criteria. *Computers & Operations Research, 20*(1), 1–14.
Wuttipornpun, T., & Yenradee, P. (2014). Finite capacity material requirement planning system for assembly flow shop with alternative work centres. *International Journal of Industrial and Systems Engineering, 18*(1), 95–124.
Yan, H.-S., Wan, X.-Q., & Xiong, F.-L. (2015). Integrated production planning and scheduling for a mixed batch job-shop based on alternant iterative genetic algorithm. *Journal of the Operational Research Society, 66*(8), 1250–1258.
Yan, H.-S., Wang, H.-X., & Zhang, X.-D. (2013). Simultaneous batch splitting and scheduling on identical parallel production lines. *Information Sciences, 221*, 501–519.
Yan, H.-S., Wang, N.-S., Cui, X.-Y., & Zhang, J.-G. (1997). Modeling, scheduling and control of flexible manufacturing systems by extended high-level evaluation Petri nets. *IIE Transactions, 29*(2), 147–158.
Yan, H.-S., Wang, N.-S., Zhang, J.-G., & Cui, X.-Y. (1998). Modelling, scheduling and simulation of flexible manufacturing systems using extended stochastic high-level evaluation Petri nets. *Robotics and Computer-Integrated Manufacturing, 14*(2), 121–140.
Yan, H.-S., Xia, Q.-F., Zhu, M.-R., Liu, X.-L., & Guo, Z.-M. (2003). Integrated production planning and scheduling on automobile assembly lines. *IIE Transactions, 35*(8), 711–725.
Yan, H.-S., & Zhang, X.-D. (2007). A case study on integrated production planning and scheduling in a three-stage manufacturing system. *IEEE Transactions on Automation Science and Engineering, 4*(1), 86–92.
Yang, H.-B., & Yan, H.-S. (2009). An adaptive approach to dynamic scheduling in knowledgeable manufacturing cell. *The International Journal of Advanced Manufacturing Technology, 42*(3–4), 312.
Zhang, X.-D., & Yan, H.-S. (2005). Integrated optimization of production planning and scheduling for a kind of job-shop. *The International Journal of Advanced Manufacturing Technology, 26*(7–8), 876–886.